

Dokumentation: Foistlinge Vorbestell-Tool

Stand: 18. September 2026

1. Was ist das

Ein schlankes Web-Tool für die Vorbestellung des Vinyl-Albums **"Hymnen für Trinker"** von **Foistlinge**. Besucher scannen einen QR-Code, landen auf einer Landingpage, bezahlen direkt per PayPal (24,00 € inkl. 6,00 € Versand) und PayPal fragt automatisch die Lieferadresse ab. Jede bezahlte Bestellung wird in einer Datenbank gespeichert; ein passwortgeschützter Admin-Bereich zeigt Verkaufszahl, Umsatz und alle Bestellungen an und kann sie als Excel-Datei exportieren.

2. Live-Status

Landingpage	https://foistlinge-vorbestellung.onrender.com
Admin-Bereich	https://foistlinge-vorbestellung.onrender.com/admin
QR-Code	https://foistlinge-vorbestellung.onrender.com/qr.png (zeigt automatisch auf die aktuelle Domain)
Code-Repository	https://github.com/snatch666/foistlinge-vorbestellung
Hosting	Render.com (kostenloser Web-Service-Plan)
Datenbank	Neon.tech (Postgres, Region Frankfurt/eu-central-1)
Zahlungsanbieter	PayPal – Live-Modus aktiv (echtes Geld wird abgebucht)

Admin-Zugangsdaten (Benutzername/Passwort) und alle API-Schlüssel stehen **nicht** in dieser Dokumentation, sondern nur in den Render-Umgebungsvariablen und in der lokalen `.env`-Datei (siehe Abschnitt 7).

3. Funktionsübersicht

- **Landingpage** (/): Albumcover, Titel, Tracklist, Preis, PayPal-Button.
- **PayPal-Checkout**: Bezahlung läuft komplett über PayPal (PayPal-Guthaben, Debit-/Kreditkarte). PayPal fragt beim Bezahlvorgang selbst die Versandadresse des Käufers ab – dafür gibt es kein eigenes Adressformular auf der Seite.
- **Automatische Speicherung**: Nach erfolgreicher Zahlung wird die Bestellung (Name, E-Mail, Adresse, Betrag, PayPal-Order-ID, Datum) in der Datenbank gespeichert.

- **Admin-Bereich** (/admin , passwortgeschützt):
- Anzahl verkaufter Platten und Gesamtumsatz auf einen Blick
- Tabelle aller Bestellungen mit Adresse
- Button "Als Excel exportieren" (.xlsx-Download für Versand/Buchhaltung)
- Anzeige des QR-Codes zum Ausdrucken (zeigt immer auf die aktuell aufgerufene Domain, unabhängig davon wo/wie oft neu deployed wird)

4. Technischer Aufbau

Stack: Node.js (Express) als Server, reines HTML/CSS/JavaScript im Frontend (keine Frameworks), PayPal REST API v2, Postgres (Neon) für die Datenspeicherung.

Foistlinge VVK/	
- server.js	Express-Server, alle Routen
- lib/	
- paypal.js	PayPal-API-Anbindung (Order erstellen/erfassen)
- orders.js	Datenspeicherung (Postgres, mit Datei-Fallback)
- auth.js	Basic-Auth-Schutz für /admin
- public/	Landingpage (öffentlich)
- index.html	
- style.css	
- app.js	Lädt PayPal-SDK, steuert Bezahlvorgang
- public-admin/	Admin-Oberfläche (nur mit Login erreichbar)
- index.html	
- admin.css	
- admin.js	
- data/orders.json	Nur Fallback-Speicher, falls keine DATABASE_URL gesetzt ist
- .env	Zugangsdaten & Konfiguration (NICHT im Git-Repo)
- .env.example	Vorlage für .env
- package.json	

Datenfluss einer Bestellung

1. Besucher klickt PayPal-Button -> Server erstellt eine PayPal-Order (`POST /api/paypal/create-order`).
2. Besucher bezahlt bei PayPal, gibt Versandadresse ein.
3. Frontend ruft `POST /api/paypal/capture-order/:orderID` auf -> Server bestätigt die Zahlung bei PayPal, liest Name/E-Mail/Adresse aus der PayPal-Antwort aus und speichert die Bestellung in der Datenbank.
4. Admin-Bereich liest die Datenbank aus und zeigt/exportiert die Daten.

Datenspeicherung

Ist die Umgebungsvariable `DATABASE_URL` gesetzt (aktuell: ja, Neon-Postgres), speichert `lib/orders.js` jede Bestellung in der Tabelle `orders` dieser Datenbank – dauerhaft, unabhängig von

Server-Neustarts. Ist sie nicht gesetzt, nutzt die App automatisch `data/orders.json` als einfachen Datei-Speicher (praktisch für lokale Tests, nicht für den Dauerbetrieb gedacht, da manche Hosters diese Datei bei jedem Neustart zurücksetzen).

5. Admin-Bereich nutzen

1. `https://foistlinge-vorbestellung.onrender.com/admin` öffnen.
2. Der Browser fragt nach Benutzernamen/Passwort (siehe `.env` bzw. Render-Umgebungsvariablen `ADMIN_USER` / `ADMIN_PASS`).
3. Oben: Anzahl verkaufter Platten + Umsatz.
4. Darunter aufklappbar: der QR-Code zum Ausdrucken für die Veranstaltung.
5. Tabelle: alle Bestellungen mit Adresse, neueste zuerst.
6. Button "**Als Excel exportieren**": lädt eine `.xlsx`-Datei mit allen Bestellungen herunter – praktisch für den Versand nach der Veranstaltung.

6. Preis / Produktdaten ändern

In den Render-Umgebungsvariablen (oder lokal in `.env`) anpassen:

- `ALBUM_PRICE` – Albumpreis ohne Versand (aktuell 18.00)
- `ALBUM_SHIPPING` – Versandkosten (aktuell 6.00)
- `ALBUM_CURRENCY` – Währung (aktuell EUR)

Angezeigt und über PayPal abgebucht wird immer die Summe aus Preis und Versand. Nach einer Änderung in Render speichert die Plattform automatisch und deployed neu.

Weitere Inhalte (Titel, Tracklist, Cover-Bild, Texte) stehen direkt in `public/index.html` und können dort angepasst werden.

7. Zugangsdaten & Konfiguration

Alle Zugangsdaten liegen ausschließlich in:

- **lokal**: der Datei `.env` im Projektordner (nicht im Git-Repo, siehe `.gitignore`)
- **live**: den Environment Variables im Render-Dashboard des Web-Service

Verwendete Variablen (siehe auch `.env.example`):

Variable	Bedeutung
<code>PAYPAL_CLIENT_ID</code> / <code>PAYPAL_CLIENT_SECRET</code>	Zugangsdaten der PayPal-App (developer.paypal.com)
<code>PAYPAL_ENV</code>	<code>sandbox</code> (Testmodus) oder <code>live</code> (echtes Geld) – aktuell live

Variable	Bedeutung
<code>DATABASE_URL</code>	Connection-String der Neon-Postgres-Datenbank
<code>ADMIN_USER</code> / <code>ADMIN_PASS</code>	Login für den <code>/admin</code> -Bereich
<code>ALBUM_PRICE</code> / <code>ALBUM_SHIPPING</code> / <code>ALBUM_CURRENCY</code>	Produktdaten
<code>PORT</code>	Server-Port (lokal 3000, auf Render automatisch gesetzt)

8. Verwendete Konten / Dienste

Für den Betrieb wurden folgende (kostenlose) Konten angelegt und müssen für Änderungen am Dienst verwaltet werden:

- **GitHub** (github.com/snatch666) – Code-Repository
- **Render** (render.com) – Hosting des Node-Servers, kostenloser Plan
- **Neon** (neon.tech) – Postgres-Datenbank, kostenloser Plan
- **PayPal Developer** (developer.paypal.com) – App für die PayPal-Anbindung, verknüpft mit dem PayPal-Business-Konto, auf das die Zahlungen fließen

9. Bekannte Einschränkungen

- **Render Free Plan:** Der Server "schläft" nach ca. 15 Minuten Inaktivität ein. Der nächste Aufruf dauert dann ca. 30–50 Sekunden länger (einmaliges "Aufwachen"). Für den Veranstaltungstag i. d. R. unkritisch, da regelmäßig Traffic kommt.
- **PayPal-App-Öffnung auf dem Handy:** Ob sich beim Bezahlen die native PayPal-App öffnet oder alles im Browser bleibt, entscheidet iOS/PayPal selbst (abhängig von Browser, ob die App installiert/eingeloggt ist). Kein Fehler unsererseits, die Bezahlung funktioniert in beiden Fällen gleich zuverlässig.
- **Admin-Schutz:** Einfacher Benutzername/Passwort-Schutz (HTTP Basic Auth) über HTTPS. Ausreichend für ein rudimentäres internes Tool, aber ohne Schutz gegen wiederholtes Passwort-Erraten (kein Rate-Limiting) und ohne Zwei-Faktor-Login.
- **Rechtliches (Datenschutz/Impressum):** Aktuell fehlen eine Datenschutzerklärung und ein Impressum auf der Seite – für eine öffentliche Verkaufsseite in Deutschland eigentlich Pflicht. Siehe Abschnitt 10.

10. Rechtliches / Datenschutz (offene Punkte)

Diese Seite verarbeitet personenbezogene Daten (Name, Adresse, E-Mail über PayPal) und ist damit an die DSGVO gebunden. Aktuell **noch nicht final umgesetzt**:

- Impressum (§5 TMG/DDG-Pflicht für gewerbliche Seiten)
- Datenschutzerklärung (Art. 13 DSGVO): welche Daten, wozu, wie lange gespeichert, wer Zugriff hat (PayPal, Neon, Render als Auftragsverarbeiter)
- Löschkonzept für Bestelldaten nach Versand/Aufbewahrungsfrist
- Ggf. Auftragsverarbeitungsverträge mit Neon/Render prüfen (Neon-Datenbank läuft aktuell in der EU-Region Frankfurt)

Hinweis: Dies ist keine Rechtsberatung. Vor dem breiten Einsatz der Seite empfiehlt sich eine kurze Prüfung durch einen Anwalt oder Verband, bzw. die Nutzung eines Generators wie e-recht24.de für Impressum/Datenschutzerklärung.

11. Wartung / nach der Veranstaltung

- **Bestellungen sichern:** Im Admin-Bereich regelmäßig "Als Excel exportieren" nutzen, spätestens direkt nach der Veranstaltung.
- **Zurück auf Sandbox:** Falls nach der Veranstaltung keine weiteren Bestellungen mehr angenommen werden sollen, entweder den Render-Service pausieren/löschen oder `PAYPAL_ENV` zurück auf `sandbox` setzen.
- **Admin-Passwort ändern:** Bei Bedarf jederzeit `ADMIN_PASS` in Render ändern.
- **Kosten im Blick behalten:** Render und Neon laufen aktuell auf kostenlosen Plänen – bei stark steigendem Traffic ggf. Limits prüfen.

12. Lokale Entwicklung

```
npm install
npm start
```

Voraussetzung: Node.js Version 18 oder neuer und eine ausgefüllte `.env`-Datei (siehe `.env.example`). Details zur Ersteinrichtung (PayPal-App, Neon-Datenbank, GitHub, Render) stehen in der [README.md](#).